Contents

Complete Build and Test Guide — Land Administration Framework **1**
Development Philosophy . . . 1
Environment and Repository Setup . . . 2
Component 1 — GIS-Based Spatial Verification (Objective i) . . . 2
Component 2 — Blockchain NFT Tokenisation and Statutory Revocation (Objective ii) 3
Part A — Smart contract execution (issuing/updating records) . . . 3
Part B — NFT title issuance . . . 3
Part C — Statutory revocation (the piece most likely still to be built) . . . 4
Component 3 — Community Verification Oracle and MQTT Communication (Objective iii) . . . 6
Part A — Ownership verification (the oracle) . . . 6
Part B — MQTT communication and synchronisation . . . 6
Integration — Unified Registration Workflow . . . 8
Component 4 — Cross-Cutting Evaluation (Objective iv) . . . 8
Define the conventional baseline first . . . 8
Part A — Fraud and tampering scenarios . . . 8
Part B — Processing latency (5-stage timing) . . . 9
Part C — Storage overhead . . . 10
Build Order Summary . . . 11

Complete Build and Test Guide — Land Administration Framework

System: An integrated Blockchain–GIS–NFT–Community Verification Oracle–MQTT framework for land title registration, verification, and lawful revocation.

How this document is organised: Everything needed to build, test, and report results is inline below — no external chapters or documents to cross-check. Four components, each built and tested in isolation first, then integrated, then evaluated as a whole. For each component you'll find: what it does, the exact steps it must follow, the code to start from, how to test it, and a blank results table showing exactly what the finished results should look like.

Development Philosophy

Four rules govern the build order:

1. **One component, one phase, one validation gate.** Don't start building the next component until the current one passes its own isolated test — don't debug a fully-wired system with five unknowns at once.
2. **Every component gets its success criteria defined before it's built.**
3. **The step-by-step procedures below are the spec, not a suggestion** — implement them in the stated order.
4. **Cross-cutting evaluation (security, fraud resistance, storage, latency) happens once, last, across the whole integrated system** — never mixed into an individual component's own test.

Environment and Repository Setup

Goal: A running skeleton with every service reachable, before any business logic exists.

Repository structure:

```
monorepo/
├── apps/
│   ├── web/              # Next.js
│   ├── mobile/           # React Native (Expo)
│   └── backend/          # NestJS
├── contracts/            # Solidity smart contracts
├── docker-compose.yml
└── infra/
    └── besu/              # Hyperledger Besu network config
```

Technology stack: - **Frontend:** Next.js (web), React Native via Expo (mobile) - **Backend:** NestJS (Node.js/TypeScript) - **Database:** PostgreSQL with the PostGIS spatial extension - **Blockchain:** Hyperledger Besu, permissioned network - **Smart contracts:** Solidity, developed/tested via Remix IDE or Hardhat - **Messaging:** MQTT broker — EMQX or Eclipse Mosquitto - **Caching/queues:** Redis - **Object storage:** MinIO (for land documents) - **Containerisation:** Docker Compose - **Version control:** Git

Blockchain consensus configuration: a hybrid Proof-of-Authority–Quorum Byzantine Fault Tolerance–Delegated Proof-of-Stake (PoA–QBFT–DPoS) mechanism. Configure Besu with: - QBFT block period: **1 second** - Epoch length: **30,000 blocks**

This combination was chosen over two alternatives: standard Proof-of-Work was rejected because its computational overhead and probabilistic finality don't suit a permissioned, institutionally governed context where validator identity is already known; standard unweighted Proof-of-Authority was rejected because it doesn't support the reputation-based, statutory-role differentiation needed (specifically, restricting the revocation function to one authorised role). QBFT gives deterministic finality suited to statutory transactions; the DPoS component allows validator weighting to reflect institutional role.

Setup tasks: 1. Scaffold the four workspaces (`npx create-next-app`, `npx react-native init` or `expo init`, `nest new`, and a bare Solidity/Hardhat project for `contracts/`). 2. Write `docker-compose.yml` bringing up PostgreSQL+PostGIS, Redis, MinIO, the MQTT broker, and a Besu node. 3. Generate the Besu QBFT genesis file with the block period and epoch length above. 4. Confirm every service is reachable: `docker compose ps` shows all healthy, NestJS connects to Postgres, and a trivial Solidity contract deploys to the local Besu node.

Validation gate: all services start clean with `docker compose up`, and a test transaction confirms on the Besu network. Don't proceed until this is reliable.

Component 1 — GIS-Based Spatial Verification (Objective i)

What it does: Validates that a submitted land parcel has correct geometry and doesn't overlap an already-registered parcel, before the application proceeds any further.

Exact procedure to implement: 1. Receive parcel coordinates submitted by the applicant. 2. Convert the coordinates into a GIS polygon. 3. Validate the geometry of the polygon (is it a well-formed, non-self-intersecting shape?). 4. Compare the polygon against existing land parcels already in the spatial database. 5. Detect any boundary overlap or topological inconsistency. 6. If the parcel satisfies all spatial validation rules, approve it for the next stage. 7. Otherwise, reject the application and return specific diagnostic feedback to the applicant (which rule failed).

Implementation notes: - Use PostGIS's `ST_IsValid()` for step 3 and `ST_Intersects()` for steps 4–5. - Reference coordinates to WGS 84 (EPSG:4326). - Define an explicit overlap tolerance buffer rather than relying on exact-geometry intersection, since real survey boundaries rarely align to the millimetre: `sql SELECT ST_Intersects(ST_Buffer(new_parcel.geom, 0.5), existing.geom) FROM parcels existing WHERE existing.id != new_parcel.id;` (0.5 = half a metre; adjust to whatever is defensible for your survey data's accuracy class, and record whatever value you use.)

How to test it (isolated, before touching any other component): Construct a test set of parcels by hand — some clean, some deliberately self-intersecting, some overlapping a seeded existing parcel — and confirm the module classifies every one correctly.

What the finished result should look like:

Table — Spatial Verification Test Results

Validation Outcome	Frequency	Percentage
Valid and non-overlapping	[n]	[%]
Invalid geometry	[n]	[%]
Overlapping existing parcel	[n]	[%]
Total	[N]	**100%**

Run this twice: once on a clean/controlled test set, once on a deliberately messier set (near-miss boundaries, slivers, slightly rotated overlaps) to see whether accuracy holds up under harder conditions. Report both tables, plus the overlap tolerance value used.

Component 2 — Blockchain NFT Tokenisation and Statutory Revocation (Objective ii)

What it does: Issues approved land titles as immutable, traceable digital tokens, and provides a legally gated mechanism for those titles to be lawfully revoked by an authorised authority — distinguishing lawful, authority-invoked mutability from unauthorised tampering.

Part A — Smart contract execution (issuing/updating records)

Exact procedure to implement: 1. Receive the verified transaction request (i.e., an application that has already passed spatial verification and ownership verification). 2. Validate the transaction against predefined business rules. 3. Execute the corresponding smart contract function. 4. Record the transaction on the permissioned blockchain. 5. Update the ownership records. 6. Generate a transaction confirmation.

Part B — NFT title issuance

Exact procedure to implement: 1. Receive the approved land registration record. 2. Generate a unique ERC-721 NFT representing the registered land parcel. 3. Associate the NFT with the verified ownership information. 4. Record the NFT reference on the blockchain. 5. Update the off-chain database with the NFT identifier. 6. Issue the digital land title to the registered owner.

Implementation notes: Build two contracts. `LandTitleNFT` extends the ERC-721 standard to generate unique digital titles. `LandRegistry` manages issuance and registry-controlled transfers. Owner-initiated transfer functions must be **disabled** — land ownership changes

should only occur through the authorised registry process, never a direct peer-to-peer NFT transfer as in a conventional marketplace. Each NFT should contain: a unique token identifier, a link to the corresponding land parcel, the blockchain transaction hash, a spatial information hash, and title metadata.

Part C — Statutory revocation (the piece most likely still to be built)

Exact procedure to implement: 1. Receive the revocation request and verify the requester's authorised governmental role. 2. Validate the revocation grounds — under Nigeria's Land Use Act (1978), Section 28, revocation is exercised by the Governor on one of two grounds: overriding public interest, or breach of a statutory condition. This is a distinct executive act from a court adjudicating an ownership dispute between two parties — a judicial dispute-resolution mechanism does **not** satisfy this requirement, even if it also results in a title change. 3. Invoke the smart contract's revocation function against the target NFT. 4. Record the revocation event immutably on the blockchain, as a **distinct event from the original issuance record** — don't just flip an ownership field, since the revocation must remain auditable as its own historical event. 5. Update the off-chain database and GIS layer to reflect the parcel's reverted status. 6. Notify the affected titleholder and the relevant registry office.

Code to implement this:

```
// Add to LandTitleNFT.sol
// Requires: import "@openzeppelin/contracts/access/AccessControl.sol";
// and: contract LandTitleNFT is ERC721, AccessControl { ... }

bytes32 public constant GOVERNOR_ROLE = keccak256("GOVERNOR_ROLE");

enum RevocationGround { OverridingPublicInterest, BreachOfStatutoryCondition }

mapping(uint256 => bool) public isRevoked;

event TitleRevoked(
    uint256 indexed tokenId,
    RevocationGround ground,
    address indexed revokedBy,
    uint256 timestamp
);

function revoke(uint256 tokenId, RevocationGround ground)
    external
    onlyRole(GOVERNOR_ROLE)
{
    require(_exists(tokenId), "LandTitleNFT: token does not exist");
    require(!isRevoked[tokenId], "LandTitleNFT: already revoked");
    isRevoked[tokenId] = true;
    emit TitleRevoked(tokenId, ground, msg.sender, block.timestamp);
}

// Block any transfer of a revoked token
// (OpenZeppelin v4 hook name; use _update() override on OpenZeppelin v5)
function _beforeTokenTransfer(
    address from,
    address to,
    uint256 tokenId,
```

```
    uint256 batchSize
) internal override {
    require(!isRevoked[tokenId], "LandTitleNFT: token is revoked");
    super._beforeTokenTransfer(from, to, tokenId, batchSize);
}
```

Deployment step: grant `GOVERNOR_ROLE` to one designated address (ideally a multisig, a single test address is fine for a prototype):

```
_grantRole(GOVERNOR_ROLE, governorAddress);
```

How to test it (isolated, using mock "already approved" application data — no GIS or oracle dependency needed):

```
describe("Statutory revocation", () => {
  it("allows the Governor-role address to revoke a title", async () => {
    // ground 0 = OverridingPublicInterest
    await landTitleNFT.connect(governor).revoke(tokenId, 0);
    expect(await landTitleNFT.isRevoked(tokenId)).to.equal(true);
  });

  it("reverts when a non-Governor address attempts to revoke", async () => {
    await expect(
      landTitleNFT.connect(randomUser).revoke(tokenId, 0)
    ).to.be.reverted;
  });

  it("reverts any transfer attempt on a revoked token", async () => {
    // ground 1 = BreachOfStatutoryCondition
    await landTitleNFT.connect(governor).revoke(tokenId, 1);
    await expect(
      landTitleNFT.connect(owner).transferFrom(owner.address, buyer.address, tokenId)
    ).to.be.reverted;
  });
});
```

What the finished result should look like:

Table A — Smart Contract and NFT Lifecycle Events

Blockchain Event	Frequency
NFT land titles issued	[n]
Registry-controlled transfers completed	[n]
Owner-initiated transfer attempts rejected	[n]

Table B — Statutory Revocation Test Results

Test	Result	Notes
Governor-role revocation succeeds	[Pass/Fail]	
Non-Governor revocation attempt reverts	[Pass/Fail]	

Test	Result	Notes
Transfer of a revoked token reverts	`[Pass/Fail]`	
Gas cost of `revoke()` call	`[n]` gas	
Confirmation time of `revoke()` call	`[n]` ms	this value is also the `T_contract` figure needed for Component 4's timing table

---

Component 3 — Community Verification Oracle and MQTT Communication (Objective iii)

Part A — Ownership verification (the oracle)

What it does: Checks submitted ownership information against institutional and community sources before anything reaches the blockchain, reconciling statutory and customary tenure claims.

Exact procedure to implement: 1. Receive ownership information submitted during land registration. 2. Forward the information to the verification service. 3. Retrieve ownership records from authorised institutional and community sources. 4. Compare the retrieved information with the submitted records. 5. If the ownership information is successfully verified, approve it for the next stage. 6. Otherwise, reject the request and return the verification result for administrative review.

Implementation notes: Build this as an intermediary NestJS service. Document precisely what counts as a "match" versus a "conflict" in your comparison logic — this rule is what later gets stress-tested in Component 4's fraud scenarios, so vague matching criteria now becomes an untestable claim later.

How to test it (isolated, with GIS and blockchain both mocked out):

Table — Ownership Verification Test Results

Verification Outcome	Frequency	Percentage
Successfully verified	`[n]`	`[%]`
Incomplete supporting documents	`[n]`	`[%]`
Ownership information conflict	`[n]`	`[%]`
Legal verification pending	`[n]`	`[%]`
Total	`[N]`	**100%**

Part B — MQTT communication and synchronisation

What it does: Publishes real-time notifications across backend, web, and mobile clients whenever a significant workflow event occurs, and supports offline data capture with later synchronisation for low-connectivity conditions.

Exact procedure to implement: 1. Receive the validated transaction request. 2. Publish the transaction message to the MQTT broker. 3. Notify all subscribed system components. 4. Synchronise transaction information across the integrated framework. 5. Confirm successful message delivery. 6. Record the communication event for system monitoring and auditing.

Documentation task — fill this in from the actual implementation, not the original plan:

Table — MQTT Topic and Payload Schema

Topic	Payload fields	QoS level
e.g. `land/registration/status`	e.g. `{applicationId, status, timestamp}`	e.g. 1
`[...]`	`[...]`	`[...]`

Approximate the byte size of a representative payload with `JSON.stringify(payload).length` — this feeds directly into Component 4's storage-overhead calculation.

Testing under degraded connectivity (not just normal conditions):

An initial test under normal network conditions is a reasonable first pass, but the framework's whole justification for using MQTT is resilience under poor connectivity — so it must also be tested under degraded conditions, using the Linux `tc` utility to inject latency and packet loss:

```
# Identify the network interface
ip addr show

# Moderate degradation: 200ms latency, 10% packet loss
sudo tc qdisc add dev eth0 root netem delay 200ms loss 10%
npm run test:mqtt

# Remove before testing the next condition
sudo tc qdisc del dev eth0 root

# Severe degradation: 500ms latency, 25% packet loss
sudo tc qdisc add dev eth0 root netem delay 500ms loss 25%
npm run test:mqtt
sudo tc qdisc del dev eth0 root
```

Offline queue test: disable network on a test device, submit a registration through the mobile app, confirm the submission sits in a local queue (check local storage directly, not just the UI), re-enable network, confirm the queued submission reaches the backend, and log the delay. If offline queueing isn't actually implemented, build it now — a local buffer that flushes on reconnect.

What the finished result should look like:

Table — MQTT Delivery Performance by Network Condition

Condition	Messages Sent	Successfully Delivered	Delivery Rate
Normal connectivity	`[N]`	`[n]`	`[%]`
Moderate degradation (200ms / 10% loss)	`[N]`	`[n]`	`[%]`
Severe degradation (500ms / 25% loss)	`[N]`	`[n]`	`[%]`

Table — Offline Queue Test

Scenario	Result
Submission made while offline is queued locally	`[Yes/No]`
Queued submission is delivered after reconnect	`[Yes/No]`
Time from reconnect to successful delivery	`[n]` seconds

Integration — Unified Registration Workflow

Once Components 1–3 each pass their isolated tests, wire them into the single end-to-end pipeline:

Sequence: application submission → spatial verification (Component 1) → community attestation / ownership verification (Component 3, Part A) → legal verification → registrar approval → blockchain registration (Component 2, Part A) → NFT title issuance (Component 2, Part B) → MQTT notification (Component 3, Part B).

Add role-based access control across every stage (citizen, surveyor, registrar, land administrator, Governor), and log every state transition to an immutable audit trail.

What the finished result should look like:

Table — End-to-End Registration Workflow Outcomes

Application Status	Frequency	Percentage
Successfully completed (NFT issued)	[n]	[%]
Rejected during review	[n]	[%]
Cancelled by applicant	[n]	[%]
Still under processing	[n]	[%]
Total	[N]	**100%**

Run this only after Components 1–3 have individually passed their own tests — if this integration test fails, the bug is almost certainly in how two components interact, not in either component alone.

Component 4 — Cross-Cutting Evaluation (Objective iv)

What this evaluates: the fully integrated system's security, fraud resistance, storage efficiency, and processing speed — all measured the same way across both the proposed system and a defined conventional baseline, so every number has a comparator.

Define the conventional baseline first

Before running any comparison, build a simplified simulator representing a manual/paper-based process: it should check document *presence* but not authenticity, and perform no automated spatial cross-check and no automated ownership cross-check. It doesn't need to be sophisticated — it needs to represent what a non-automated process would and wouldn't catch.

Part A — Fraud and tampering scenarios

Four adversary scenarios, each tested independently — don't average them into one combined "fraud rate":

#	Scenario	How to construct the test input	What layer it targets
1	Forged or altered supporting document	Submit a registration with a doctored document (edited metadata, mismatched signature)	Application/document layer
2	Duplicate or overlapping parcel claim	Submit coordinates that intentionally overlap an already-registered parcel	Spatial/GIS layer
3	Colluding verification informant	Submit ownership info where the community/institutional attestation step is deliberately falsified	Oracle/verification layer
4	Post-revocation NFT reuse	Revoke a title (Component 2, Part C), then attempt to transact against it	Blockchain/NFT layer

Run each scenario at least 20–30 times against both the proposed system and the baseline simulator.

What the finished result should look like:

Table — Fraud Scenario Detection and Rejection Rates

Scenario	Trials	Proposed Framework Outcome	Baseline Outcome
1. Forged/altered documents	[N]	[n detected / N (%)]	[n detected / N (%)]
2. Duplicate/overlapping parcel	[N]	[n detected / N (%)]	[n detected / N (%)]
3. Colluding verification informant	[N]	[n detected / N (%)]	[n detected / N (%)]
4. Post-revocation NFT reuse	[N]	[n detected / N (%)]	[n detected / N (%)]

Part B — Processing latency (5-stage timing)

Instrument each stage of one complete registration transaction:

```
const { performance } = require('perf_hooks');

async function timedRegistration(applicationData) {
  const t0 = performance.now();
  await gisVerify(applicationData.parcel);
  const t_gis = performance.now() - t0;
```

```
  const t1 = performance.now();
  await oracleVerify(applicationData.ownership);
  const t_oracle = performance.now() - t1;

  const t2 = performance.now();
  await executeSmartContract(applicationData.transaction);
  const t_contract = performance.now() - t2;

  const t3 = performance.now();
  await issueNFT(applicationData.title);
  const t_nft = performance.now() - t3;

  const t4 = performance.now();
  await publishMQTT(applicationData.event);
  const t_sync = performance.now() - t4;

  const T_total = t_gis + t_oracle + t_contract + t_nft + t_sync;
  return { t_gis, t_oracle, t_contract, t_nft, t_sync, T_total };
}
```

Run this at least 30 times on separate transactions, and run the equivalent flow through the baseline simulator for the same transaction type.

What the finished result should look like:

Table — End-to-End Processing Latency (mean of 30+ runs)

Stage	Proposed Framework (ms)	Baseline (ms)
Spatial verification	[mean ± sd]	[mean ± sd]
Ownership/oracle verification	[mean ± sd]	[mean ± sd]
Smart contract execution	[mean ± sd]	[mean ± sd]
NFT issuance	[mean ± sd]	[mean ± sd]
MQTT synchronisation	[mean ± sd]	[mean ± sd]
Total	[mean ± sd]	[mean ± sd]
Improvement		[% reduction]

Part C — Storage overhead

Open the actual deployed contract's storage layout (and the off-chain schema for anything stored there instead) and record the real bit length used for each field:

```
S_id      = ? bits   (parcel identity hash — e.g. keccak256 = 256 bits)
S_geom    = ? bits   (spatial geometry reference/hash stored on-chain)
S_owner   = ? bits   (ownership/NFT-holder reference —
                      likely a 160-bit Ethereum address = 20 bytes)
S_oracle  = ? bits   (from the MQTT payload schema table above)
S_time    = ? bits   (timestamp — check what's actually stored,
                      not just Solidity's uint256 default)
S_sig     = ? bits   (validator/contract signature, if stored on-chain)
S_merkle  = ? bits   (Merkle proof metadata, if applicable — many
                      prototypes don't store this; record "0" if so)
```

Total per-record size = sum of the seven fields above. Multiply by the number of registered parcels for network-wide storage. Compute the same total for the baseline (e.g., a naive

design storing full geometry data on-chain instead of a hash) to get a percentage reduction.

What the finished result should look like:

Table — Storage Overhead Comparison

	Proposed Framework	Baseline
Per-record size (bytes)	`[n]`	`[n]`
Network storage for `[N]` parcels	`[n]`	`[n]`
Reduction		`[% reduction]`

Build Order Summary

Step	Component	Depends on	Must pass before moving on
0	Environment setup	—	All services healthy, test transaction confirms
1	GIS spatial verification	Step 0	Hand-constructed test parcels classify correctly
2	Blockchain/NFT/revocation	Step 0	Six-case test suite passes with mock inputs
3	Oracle + MQTT	Step 0	Verification test + normal-condition MQTT test pass
4	Integration	Steps 1–3	End-to-end workflow test passes
5	Cross-cutting evaluation	Step 4	All four result tables above completed with real data

Each row's "must pass" column is a hard gate — the single biggest reason to avoid debugging a fully-integrated system before its individual parts are each proven to work alone.